

- Nesse tipo de banco, cada **registro (documento)** fica armazenado em uma **coleção** específica sem esquema explícito;
- São fortemente orientado a agregados com uma certa estrutura básica nos conjuntos de atributos;
 - Estruturas JSON ou XML são normalmente utilizados na definição dos agregados;
- Consultas podem ser submetidas baseadas nos atributos e/ou índices dos agregados;
- É possível recuperar parte do agregado ou todo o agregado;
- Estudo de caso: **MongoDB**
 - Estrutura documentos em um formato similar ao JSON, chamado BSON;
 - Um dos bancos NoSQL mais usados no



mundo atualmente;

- Seu ambiente interativo e suas consultas são escritas em JavaScript;
- Distribuição gratuita para Linux, Mac e Windows;
- Quando usar o MongoDB?
 - No gerenciamento de sistemas;
 - Em aplicações mobile;
 - Na visualização de dados;
 - Entre outros;
- Não devemos utilizá-lo em sistemas que demandam muitos relacionamentos entre agregados e transações;



- Comparativo entre conceitos do modelo relacional e do MongoDB:

Database	Database
Tabela	Coleção
Linha	Documento (BSON)
Coluna	Campo
Index	Index
Table Join	Documentos Aninhados
Chave primária	Chave primária (_id)



- Documentos BSON são semelhantes a:



- Uma linha de uma tabela relacional;
- Um array do PHP;
- Um Map do Java;
- Um dicionário do Python;
- Um objeto JavaScript;
- Criando nosso primeiro documento:
 - O MongoDB fornece um cliente chamado **mongo**, que permite executar comandos para a manipulação de dados. Para executá-lo, digite o comando mongo em um terminal:
 - O MongoDB também possui o conceito de databases como nos bancos relacionais. Podemos listar todos os bancos existentes no MongoDB a partir do comando **show dbs**;

aluno@aluno:~\$ mongo

MongoDB shell version v4.2.3

connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb

aluno@aluno:~\$ mongo

MongoDB shell version v4.2.3

connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb

Implicit session: session { "id" : UUID("c66a7932-ef50-4335-b35e-2cf1810b1f19") }

MongoDB server version: 4.2.3

Server has startup warnings:

2020-02-01T17:24:48.042+0000 I STORAGE [initandlisten]

2020-02-01T17:24:48.042+0000 I STORAGE [initandlisten] ** WARNING: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine

2020-02-01T17:24:48.042+0000 I STORAGE [initandlisten] ** See <http://dochub.mongodb.org/core/prodnotes-filesystem>

2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten]

2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for the database.

2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten] ** Read and write access to data and configuration is unrestricted.

2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten]

Enable MongoDB's free cloud-based monitoring service, which will then receive and display metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you and anyone you share the URL with. MongoDB may use this information to make product improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()

To permanently disable this reminder, run the following command: db.disableFreeMonitoring()

> show dbs;

admin 0.000GB

config 0.000GB

local 0.000GB

only recommended with the WiredTiger storage engine

```
2020-02-01T17:24:48.042+0000 I STORAGE [initandlisten] See http://dochub.mongodb.org/core/prodnotes-filesystem
2020-02-01T17:24:54.307+0000 CONTROL [initandlisten]
2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for the database.
2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten] Read and write access to data and configuration is unrestricted.
2020-02-01T17:24:54.307+0000 I CONTROL [initandlisten]
```

Enable MongoDB's free cloud-based monitoring service, which will then receive and display metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you and anyone you share the URL with. MongoDB may use this information to make product improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: `db.enableFreeMonitoring()` To permanently disable this reminder, run the following command: `db.disableFreeMonitoring()`

```
> show dbs;
admin 0.000GB
config 0.000GB
local 0.000GB
```

- **Criando nosso primeiro documento:**
 - Para escolher um banco de dados existente, utilizamos o comando **use NOME_BANCO;**. Caso o banco não exista, será criado um banco vazio:
 - Para listar todas as coleções do banco selecionado, utilizamos o

comando **show collections**;, equivalente ao comando show tables do MySQL:

- Criando nosso primeiro documento:
 - A forma mais fácil de se criar uma coleção é criar e inserir o primeiro documento a ser armazenado em um novo nome de coleção. Para isso, usamos a função **insert** do objeto **db**, utilizando a sintaxe JavaScript. Exemplo:
 - O objeto **db** permite ao administrador ter acesso a esse e a outros métodos de manipulação de dados no MongoDB;

```
> show dbs;
admin    0.000GB
config  0.000GB
local    0.000GB
>
>
> use ligado;
switched to db ligado
```

```
> show dbs;
admin 0.000GB
config 0.000GB
local 0.000GB
> use ligado;
```

switched to db ligado

```
> show collections;
```

```
> db.albums.insert({});  
WriteResult({ "nInserted" : 1 })
```

```
show collections;  
albums
```

```
show dbs;  
admin 0.000GB  
config 0.000GB  
ligado 0.000GB  
local 0.000GB  
>
```

- **Criando nosso primeiro documento:**
- Para fazer uma busca em uma coleção, usamos a função **find**.
Exemplo:

- No MongoDB, todas as coleções possuem uma espécie de chave primária chamada de id. Como ela é gerenciada pelo próprio MongoDB, não temos controle sobre ela;

```
> show dbs;
admin    0.000GB
config  0.000GB
local    0.000GB
>
>
> use ligado;
switched to db ligado
>
> show collections;
>
> db.albuns.insert({});
WriteResult({ "nInserted" : 1 })
>
>
> show collections;
albuns
>
>
> show dbs;
admin    0.000GB
config  0.000GB
ligado    0.000GB
local    0.000GB
>
```

```
> show dbs;
admin 0.000GB
config 0.000GB
local 0.000GB
>
>
> use ligado;
switched to db ligado
>
> show collections;
>
> db.albuns.insert({});
WriteResult({ "nInserted" : 1 })
>
>
> show collections;
albuns
>
>
> show dbs;
admin 0.000GB
config 0.000GB
ligado 0.000GB
local 0.000GB
>
```

ligado 0.000GB
local 0.000GB

```
> db.albums.insert({});  
WriteResult({ "nInserted" : 1 })  
> db.albums.find({});  
  "_id" : ObjectId("5e395f2e76d8ee5dd60fa40f") { "_id" :  
    ObjectId("5e39644276d8ee5dd60fa410") }
```

- Criando nosso primeiro documento:

- O conjunto de chaves presentes nos parâmetros das funções **insert** e **find** nos permite definir campos de documentos a partir de objetos JSON;

- Resumidamente, o formato de um objeto JSON segue uma sintaxe similar ao

exemplo ao lado:

```
{  
  "nome" : "Ricardo",  
  "sobrenome" :  
    "Roberto", "idade" :  
    37,  
  "nascimento" : new  
    Date(1984, 1, 28)  
}
```

- Criando

documentos:

- Vamos inserir alguns álbuns para exemplificar a inserção de coleções com campos:
- Criando documentos:
 - Vamos inserir alguns álbuns para exemplificar a inserção de coleções com campos:

- Buscando documentos:
 - Agora vamos utilizar JSON como critério de busca (filtro) na função **find**:
- Buscando documentos:
 - Agora vamos utilizar JSON como critério de busca (filtro) na função **find**:

Similar a seguinte consulta SQL: `SELECT * FROM albuns a WHERE a.nome = "Master of Puppets"`

- Buscando documentos:

- Agora vamos utilizar JSON como critério de busca (filtro) na função **findOne** (seleciona o primeiro documento que satisfaça o filtro):
- É possível fazer buscas mais complexas no MongoDB:

\$gt	Maior que
\$gte	Maior ou igual
\$in	Está contido
\$lt	Menor que
\$lte	Menor ou igual
\$ne	Diferente
\$nin	Não está contido



- Exemplo: Buscar os álbuns com duração menor que 30 minutos, procuraremos os documentos com o campo duração menor que

1800:

- Exemplo: Buscar os álbuns com duração menor que 30 minutos, procuraremos os documentos com o campo duração menor que 1800:

Similar a consulta SQL: `SELECT * FROM`

`: 1800`

- É possível utilizar operadores lógicos no MongoDB para a criação de condições compostas:



\$and	E
\$nor	NEM
\$not	NÃO
\$or	OU

- Exemplo: Buscar os álbuns com o ano de lançamento menor que 1987 **E** maior ou igual a 1986:

Similar a consulta SQL:

```
SELECT * FROM albuns a WHERE
a.dataLancamento >= '1986-01-01 00:00:00'
and a.dataLancamento < '1987-01-01 00:00:00'
```

- Exemplo: Buscar os álbuns com o ano de lançamento menor que 1987 **E** maior ou igual a 1986:
- Exemplo: A mesma busca, mais simplificada (possível apenas em consultas com **E**):
- Para apagar um

documento, utilizamos a função **remove**.

- Exemplo:
- Para apagar um documento, utilizamos a função **remove**.
- Exemplo:

Similar a consulta SQL: `DELETE FROM albuns a WHERE a.nome = "...And Justice for All"`

- Para atualizar um documento, utilizamos a função **update**.

- Exemplo:
Vamos adicionar o campo duração no álbum “Among the Living”:

```
    "nome" : "Among the Living",
    "produtor" : "Eddie Kramer"
  }
}
>
> db.albuns.update(
...  { "nome" : "Among the Living" },
...  { "duracao" : 3013 }
... )
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
> db.albuns.findOne(
...  {
...    "nome" : "Among the Living"
...  }
... )
null
> db.albuns.find({});
{ "_id" : ObjectId("5e395f2e76d8ee5dd60fa40f") }
{ "_id" : ObjectId("5e39644276d8ee5dd60fa410") }
{ "_id" : ObjectId("5e39678376d8ee5dd60fa411"), "nome" : "Master of Puppets", "dataLancamento" : ISODate("1986-03-03T00:00:00Z"), "duracao" : 3286 }
{ "_id" : ObjectId("5e3968f376d8ee5dd60fa413"), "nome" : "Peace Sells... but Who's Buying?", "duracao" : 2172, "estudioGravacao" : "Music Grinder Studios", "dataLancamento" : ISODate("1986-09-19T00:00:00Z") }
{ "_id" : ObjectId("5e39695176d8ee5dd60fa414"), "nome" : "Reign in Blood", "dataLancamento" : ISODate("1986-10-07T00:00:00Z"), "artistaCapa" : "Larry Carroll", "duracao" : 1738 }
{ "_id" : ObjectId("5e396b5476d8ee5dd60fa415"), "duracao" : 3013 }
>
-
```

- O que aconteceu?
- Temos que ter cuidado com a função update:
 - Ela, ao incluir um campo da forma que foi feita anteriormente, sobrescreverá o documento com os novos dados (com exceção do _id);
 - Não era o que queríamos!
 - Para adicionar ou alterar um campo existente, precisamos utilizar o operador de atualização **\$set** na função update;

```

> db.albums.find({});
{ "_id" : ObjectId("5e395f2e76d8ee5dd60fa40f") }
{ "_id" : ObjectId("5e39644276d8ee5dd60fa410") }
{ "_id" : ObjectId("5e39678376d8ee5dd60fa411"), "nome" : "Master of Puppets", "dataLancamento" : ISODate("1986-03-03T00:00:00Z"), "duracao" : 3286 }
{ "_id" : ObjectId("5e3968f376d8ee5dd60fa413"), "nome" : "Peace Sells... but Who's Buying?", "duracao" : 2172, "estudioGravacao" : "Music Grinder Studios", "dataLancamento" : ISODate("1986-09-19T00:00:00Z") }
{ "_id" : ObjectId("5e39695176d8ee5dd60fa414"), "nome" : "Reign in Blood", "dataLancamento" : ISODate("1986-10-07T00:00:00Z"), "artistaCapa" : "Larry Carroll", "duracao" : 1738 }
{ "_id" : ObjectId("5e396b5476d8ee5dd60fa415"), "duracao" : 3013 }
>
> db.albums.update(
... { "_id" : ObjectId("5e396b5476d8ee5dd60fa415") },
... {
...   $set :
...   { "nome" : "Among the Living",
...     "produtor" : "Eddie Kramer" }
... }
... )
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
> db.albums.find(
... {
...   "nome" : "Among the Living"
... }
... )
{ "_id" : ObjectId("5e396b5476d8ee5dd60fa415"), "duracao" : 3013, "nome" : "Among the Living", "produtor" : "Eddie Kramer" }
> -

```

■ Exemplo:

- Criando relacionamentos entre coleções

- Para exemplificar a criação de relacionamento entre coleções, vamos criar uma outra coleção, chamada artistas:

- Criando relacionamentos entre coleções

- Para exemplificar a criação de relacionamento entre coleções, vamos criar uma outra coleção,

chamada artistas:

```
Similar a consulta SQL: INSERT INTO artistas (nome) VALUES  
("Metallica"), ("Megadeath"), ("Slayer"),  
("Anthrax");
```

- No modelo relacional, utilizamos chaves estrangeiras para criar relacionamentos entre tabelas;
- No modelo de documentos, temos que fazer algo similar.
- Como exemplo, vamos criar o campo **artista_id** na coleção álbuns, e incluir o **_id de cada artista** nos seus respectivos álbuns;
- Exemplo:
- Agora é possível, por exemplo, listar todos os álbuns de

um artista;

- Porém, para isso, precisaremos utilizar JavaScript (variáveis, condicionais e repetição) para essa demanda;
- Isso significa que a manipulação de dados considerando relacionamento entre coleções não são feitas na camada de persistência, mas na camada de aplicação;
- Exemplo: Para testar isso, vamos incluir mais álbuns no nosso banco:
- Exemplo: Para testar isso, vamos incluir

mais álbuns no
nosso banco:

- Exemplo: Agora, vamos recuperar todos os álbuns de um artista a partir do seu nome:
- Trabalhando com documentos embutidos
 - Além de strings, datas e ids, documentos permitem a inserção de outros documentos como valores de campo (subdocumentos);
 - Isso permite realizar um processo bastante comum no cenário NoSQL, chamado de **desnormalização**, afim de diminuir a necessidade de realizar várias consultas graças a existência de relacionamentos, melhorando a performance no acesso aos dados;

- O MongoDB não possui a operação JOIN como em bancos relacionais;
- Exemplo: Vamos incluir álbuns já com informações dos artistas a partir de um subdocumento:
- Exemplo: Vamos incluir álbuns já com informações dos artistas a partir de um subdocumento:
- Exemplo: Agora vamos buscar os álbuns lançados em 1998, exibindo o nome do álbum e do artista relacionado:

```
{ "_id" : ObjectId("5e3ac3e576d8ee5dd60fa420"), "nome" : "Imaginations from the Other Side", "dataLancamento" : ISODate("1995-04-04T00:00:00Z"), "duracao" : 2958, "artista" : { "nome" : "Blind Guardian" } }  
{ "_id" : ObjectId("5e3ac43c76d8ee5dd60fa421"), "nome" : "Nightfall in Middle-Earth", "dataLancamento" : ISODate("1998-04-28T00:00:00Z"), "duracao" : 3929, "artista" : { "nome" : "Blind Guardian" } }  
> -
```

- Exemplo:
Também é
possível
realizar
buscas a
partir de um

subdocumento.

Vamos buscar todos
os álbuns do artista
Blind Guardian:

- Outras operações importantes do MongoDB:

<code>db.collection.deleteOne()</code>	Exclui um documento de uma coleção
<code>db.collection.deleteMany()</code>	Excluir mais de um documento de uma coleção
<code>db.collection.count()</code>	Conta quantos documentos existem em uma coleção

- Etapa 1: Importar um dataset – O MongoDB permite o carregamento de databases já definidas em arquivos JSON;
 - Na máquina virtual utilizada em sala de aula, verifique a existência do arquivo

restaurantes.json a partir do comando **dir**;

- Confirmada sua existência, utilize o comando **mongoimport** para criar o database **atividade1** e a coleção **restaurants**:

- Entregável: No VirtualBox, clique no menu **Visualizar / Capturar Tela**. Salve uma imagem da tela após executar a importação corretamente. Entregue tal imagem como comprovante de conclusão dessa etapa;

- Etapa 1: Importar um dataset – Segue o formato básico de um documento contido na coleção **restaurants**:
- Etapa 2: Inserindo um documento em uma coleção – Insira o documento ao lado na coleção **restaurants**;

- Entregável: No VirtualBox, clique no menu **Visualizar / Capturar Tela**.
Salve uma imagem da tela após executar a consulta corretamente.
Entregue tal imagem como comprovante de conclusão dessa etapa;
- Etapa 3: Executando consultas – Execute as seguintes consultas:
 1. Listar todos os restaurantes;
 2. Encontrar todos os restaurantes do bairro (**borough**) “Manhattan”;
 3. Encontrar todos os restaurantes com CEP (**zipcode**) “10022”;
 4. Encontrar todos os restaurantes que possuam alguma nota (**grade**) “B”;
- Entregável: Arquivo texto com os comandos utilizados para cada

consulta;

- Etapa 4: Executando consultas com operadores – Execute as seguintes consultas:

1. Listar todos os restaurantes com pontuação (**score**) maior que 30; 2. Listar todos os restaurantes com pontuação (**score**) menor que 10; 3. Listar todos os restaurantes com culinária (**cuisine**) italiana (“Italian”) E com CEP “10075”;

4. Listar todos os restaurantes com culinária (**cuisine**) italiana (“Italian”) OU com CEP “10075”;

- Entregável: Arquivo texto com os comandos utilizados para cada consulta;